Algorithmique et complexité. Correction.

I) ALGORITHMIQUE :

1°) Présentation :

La notion d'algorithme est souvent associée à l'informatique, pourtant le terme algorithme vient du nom du mathématicien perse du 9e siècle Muhammad Ibn Mūsā al-Khuwārizmī. La notion d'algorithme est donc très antérieure à la création du premier ordinateur.

Mais qu'est-ce qu'un algorithme ?
Voici deux définitions trouvées dans la littérature :

- Procédure de calcul bien définie qui prend en entrée une valeur ou un ensemble de valeurs, et qui donne en sortie une valeur ou un ensemble de valeurs.
- Un algorithme est la spécification d'un schéma de calcul sous forme d'une suite finie d'opérations élémentaires obéissant à un enchainement déterminé.

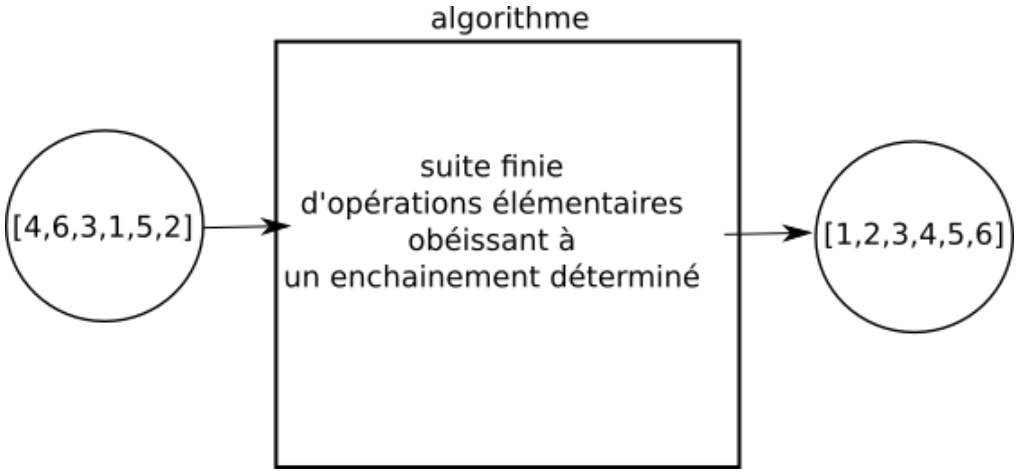
Le terme "calcul" revient dans les 2 définitions, un algorithme effectue donc des calculs (encore faudrait-il définir précisément ce terme de "calcul", mais cela nous entrainerait trop loin).

Pour concevoir un algorithme, la 2e définition nous précise qu'il est nécessaire de définir "une suite finie d'opérations élémentaires obéissant à un enchainement déterminé". L'ordre dans lequel nous allons définir nos "opérations élémentaires" va donc avoir son importance. Mais, qu'est-ce qu'une "opération élémentaire" ? Ici aussi, définir "opération élémentaire" n'est pas chose aisée ! On peut tout de même dire qu'une "opération élémentaire" est une action simple, qui doit être facilement compréhensible pour la personne chargée d'effectuer cette action. L'action "Peser 100 g de farine" peut être considérée comme une "opération élémentaire", en effet, tout le monde est capable de comprendre ce que cela signifie et la réalisation de cette opération ne prête ni à interprétation, ni à confusion. En revanche "Prendre de la farine, des œufs, du sucre et du chocolat afin de faire un gâteau au chocolat" n'est clairement pas une "opération élémentaire", car l'action décrite ici manque de précision, c'est une opération complexe (même si dans la pratique il n'est pas très difficile de faire un gâteau au chocolat, ce n'est pas évident pour quelqu'un qui n'a pas l'habitude). La frontière entre "opération élémentaire" et "opération complexe" n'est pas toujours évidente, en effet, l'opération "Peser 100 g de farine" pourrait être encore plus précise : "mettre la balance à zéro (faire la tare), verser ensuite de la farine sur le plateau de la balance jusqu'à ce que le nombre 100 soit affiché sur l'écran de la balance"

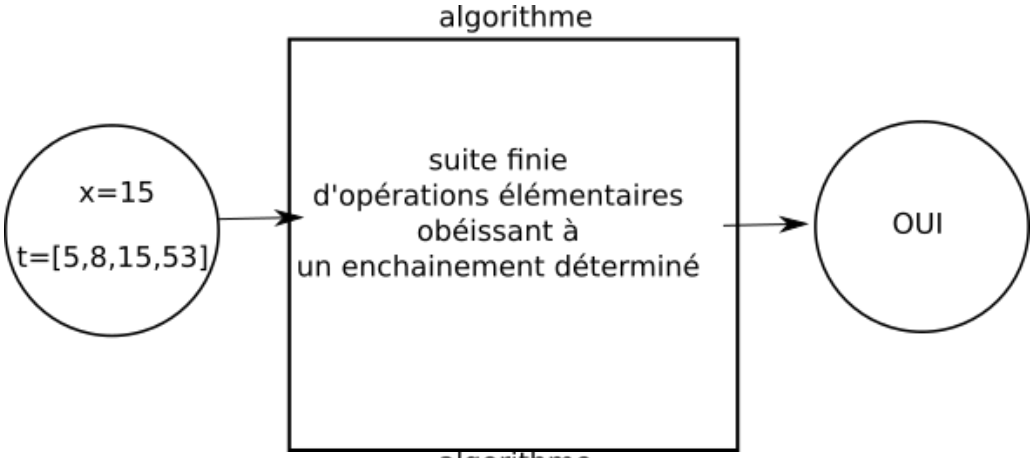
À noter que "procédure de calcul bien défini" dans la première définition est équivalent à "suite finie d'opérations élémentaires obéissant à un enchainement déterminé" dans la deuxième définition. Personnellement, je trouve la deuxième définition plus explicite.

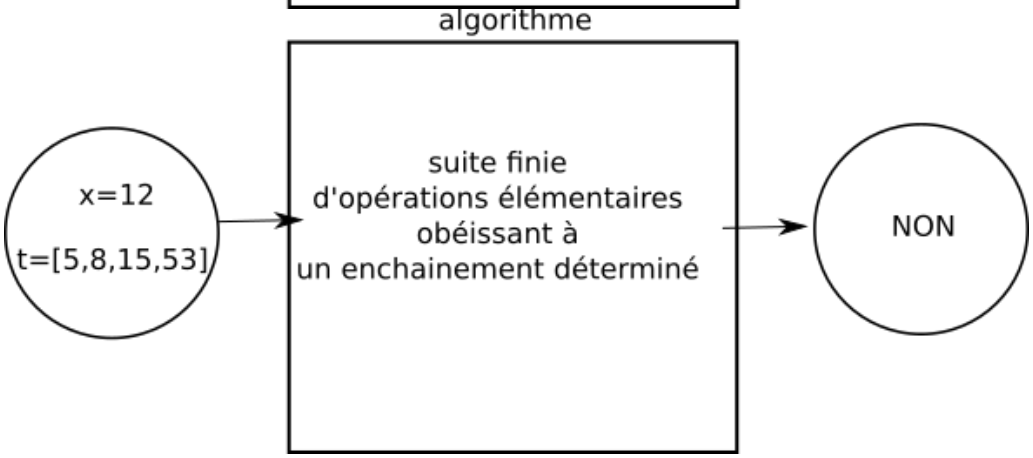
On pourrait résumer ce qu'est un algorithme par le schéma suivant :

Prenons un exemple concret :
Nous allons étudier cette année, ainsi que l'année prochaine, des algorithmes de tri pour les tableaux (un tableau ressemble beaucoup à une liste en Python, même si ce n'est pas exactement la même chose). Nous avons en entrée un tableau non trié et nous obtenons en sortie un tableau trié :



La "valeur de sortie" n'est pas obligatoirement du même type que la "valeur d'entrée". Prenons l'exemple d'un algorithme qui prend en entrée un tableau t d'entiers et un entier x, et qui "répond" par "oui" ou par "non" à la question "x est-il présent dans le tableau t ?". Dans ce cas, la "valeur de sortie" sera "oui" ou "non".





Essayons d'écrire l'algorithme décrit ci-dessus :
Nous devons trouver la suite d'opérations élémentaires qui permettra de répondre à la question : "x est-il présent dans le tableau t ?"

VARIABLES
t : tableau d'entiers
x : nombre entier
tr : booléen (VRAI ou FAUX)

```
i : nombre entier
DEBUT
tr ← FAUX
i ← 1
tant que i<=longueur(t) et que tr==FAUX:
  si t[i]==x:
    tr ← VRAI
  fin si
  i ← i+1
fin tant que
renvoyer la valeur de tr
FIN
```

Remarques :

- Quand on écrit un algorithme, on utilise un langage dit "langage naturel" ("tant que", "si"...), ce langage naturel permet de passer facilement à un langage de programmation (Python, Java...), on dit alors que l'on **implémente** l'algorithme.
- Il se peut, lorsque l'on écrit un algorithme, que le premier élément d'un tableau ait pour indice 1 (alors que dans la plupart des langages de programmation le premier élément d'un tableau a pour indice 0). Il faut donc faire attention à cela lorsque l'on veut implémenter un algorithme.
- Dans l'algorithme ci-dessus, on part du principe qu'il existe une fonction "longueur" qui prend en paramètre un tableau et qui renvoie le nombre d'éléments présents dans ce tableau. Vous noterez que déterminer le nombre d'éléments présents dans un tableau n'est pas vraiment une "opération élémentaire", pourtant ici, on considère l'utilisation de "longueur" comme une opération élémentaire ! Il arrive relativement souvent que l'on s'autorise ce genre de liberté quand on écrit un algorithme.

La première chose à faire quand on étudie un algorithme, c'est de le "faire tourner à la main" : on "exécute" l'algorithme en utilisant uniquement une feuille et un crayon. Voilà ce que cela pourrait donner avec l'algorithme que nous venons d'écrire :

t = [5,8,15,23]
x= 15

i=1
tr=FAUX
i<=4 et tr est FAUX => on entre dans la boucle
t[1] (c'est à dire 5) n'est pas égal à 15 => on n'entre pas dans le si
i←i+1 (i=2)
retour au début de la boucle

↓

i=2
tr=FAUX
i<=4 et tr est FAUX => on entre dans la boucle
t[2] (c'est à dire 8) n'est pas égal à 15 => on n'entre pas dans le si
i←i+1 (i=3)
retour au début de la boucle

↓

i=3
tr=FAUX
i<=4 et tr est FAUX => on entre dans la boucle
t[3] (c'est à dire 15) est égal à 15 => on entre dans le si
tr←VRAI
i←i+1 (i=4)
retour au début de la boucle

↓

i=4
tr=VRAI
tr est VRAI => on n'entre pas dans la boucle
renvoyer VRAI

<u>Exercice</u> :
Faire "tourner à la main" l'algorithme "x est-il présent dans le tableau t ?" avec t = [5,8,15,53] et x = 12.
t = [5,8,15,53]
x = 12

i = 1
tr = FAUX
i<=4 et tr est FAUX => on entre dans la boucle
t[1] (c'est-à-dire 5) n'est pas égal à 12 => on n'entre pas dans le si
i ← i + 1 (i = 2)
retour au début de la boucle

i = 2
tr = FAUX
i<=4 et tr est FAUX => on entre dans la boucle
t[2] (c'est-à-dire 8) n'est pas égal à 12 => on n'entre pas dans le si
i ← i + 1 (i = 3)
retour au début de la boucle

i = 3
tr = FAUX

i<=4 et tr est FAUX => on entre dans la boucle
t[3] (c'est-à-dire 15) n'est pas égal à 12 => on n'entre pas dans le si
i ← i + 1 (i = 4)
retour au début de la boucle

i = 4
tr = FAUX
i<=4 et tr est FAUX => on entre dans la boucle
t[4] (c'est-à-dire 53) n'est pas égal à 12 => on n'entre pas dans le si
fin du tant que (car i = longueur de t)
renvoyer la valeur de tr (c'est-à-dire FAUX)

2°) Complexité d'un algorithme, première approche :

La notion de complexité d'un algorithme va rendre compte de l'efficacité de cet algorithme. Pour un même problème, par exemple trier un tableau, il existe plusieurs algorithmes, certains algorithmes sont plus efficaces que d'autres (par exemple un algorithme A mettra moins de temps qu'un algorithme B pour résoudre exactement le même problème, sur la même machine).

Il existe 2 types de complexité : une complexité en temps et une complexité en mémoire. Nous nous intéresserons ici uniquement à la complexité en temps. La complexité en temps est directement liée au nombre d'opérations élémentaires qui doivent être exécutées afin de résoudre un problème donné.

L'évaluation de ce nombre d'opérations élémentaires n'est pas chose facile, on rencontre souvent des cas litigieux. Prenons tout de suite un exemple avec notre algorithme "x est-il présent dans le tableau t ?".

Il y a 2 cas à traiter :

- L'entier recherché est bien présent dans le tableau, il se trouve à la position d'index j
- L'entier recherché n'est pas présent dans le tableau

Commençons par le premier cas :

```
VARIABLES
t : tableau d'entiers
x : nombre entier
tr : booléen (VRAI ou FAUX)
i : nombre entier
DEBUT
1 fois   tr ← FAUX
1 fois   i ← 1
j+1 fois tant que i<=longueur(t) et que tr==FAUX:
j fois      si t[i]==x:
1 fois       tr ← VRAI
          fin si
j fois     i ← i+1
       fin tant que
1 fois  renvoyer la valeur de tr
FIN
```

Au total nous avons : 1 + 1 + j + 1 + j + 1 + j + 1 = 3j + 5 opérations élémentaires.

Dans le cas où l'entier recherché ne se trouve pas dans le tableau (et que le nombre d'éléments dans le tableau est n) :

```
VARIABLES
t : tableau d'entiers
x : nombre entier
tr : booléen (VRAI ou FAUX)
i : nombre entier
DEBUT
1 fois   tr ← FAUX
1 fois   i ← 1
n+1 fois tant que i<=longueur(t) et que tr==FAUX:
n fois      si t[i]==x:
```

```
0 fois      tr ← VRAI
          fin si
n fois     i ← i+1
       fin tant que
1 fois  renvoyer la valeur de tr
FIN
```

Au total nous avons : 1 + 1 + n + 1 + n + 0 + n + 1 = 3n + 4 opérations élémentaires.

Comme dans la plupart des cas n > j, on effectue plus d'opérations élémentaires quand le nombre recherché n'est pas dans le tableau (sauf dans le cas précis où l'entier recherché est en dernière position, mais nous ne tiendrons pas compte de ce cas). On parle de "complexité dans le pire des cas" quand on s'intéresse uniquement au cas où le nombre d'opérations élémentaires est le plus grand. Dans la suite nous nous intéresserons uniquement à cette complexité dans le pire des cas (dans la suite "complexité" = "complexité en temps dans le pire des cas"). Pour notre exemple, nous considérerons uniquement le cas où le nombre total d'opérations élémentaires est de 3n + 4.

Nous venons de voir que la complexité dépend de la taille du tableau, plus le tableau est grand et plus le nombre d'opérations élémentaires à effectuer est important. Pour effectuer des comparaisons entre plusieurs algorithmes, nous allons raisonner sur des tableaux de grande taille, car plus les tableaux sont grands et plus les différences entre les algorithmes seront flagrantes. Pour comparer des algorithmes, nous allons donc uniquement nous intéresser à ce que l'on appelle "l'ordre de grandeur asymptotique". La définition précise de cet "ordre de grandeur asymptotique" est trop complexe pour être abordé ici. Vous devez juste savoir que cet "ordre de grandeur asymptotique" concerne les cas où l'on prend n très très grand. On note cet "ordre de grandeur asymptotique" avec un O majuscule. Pour le cas qui nous intéresse, nous aurons : 3n + 4 = O(n)

La relation ci-dessus signifie que "3n + 4 est dominée asymptotiquement par n", autrement dit "si n est suffisamment grand, il existe une constante c qui permettra d'avoir c.n ⩾ 3n + 4", vous n'avez pas tout compris ? Cela n'a pas vraiment d'importance, ce qu'il faut retenir c'est que nous utiliserons systématiquement cette notation O pour exprimer la complexité des algorithmes : au final on dira donc que la complexité de notre algorithme "x est-il présent dans le tableau t ?" est O(n).

Comment obtient-on cette notation O à partir du nombre d'opérations élémentaires ?
Ici nous avons simplement supprimé la constante (4) et le coefficient devant le n (c'est-à-dire 3), il reste donc uniquement n d'où le 3n + 4 = O(n). Dans le cas où nous avons un polynôme de degrés quelconque, par exemple pour $6n^2 + 3n + 10$, il suffit de :

- supprimer la constante,
- garder uniquement le n qui possède l'exposant le plus grand,
- supprimer le coefficient devant le n.

On aura donc $6n^2 + 3n + 10 = O(n^2)$: "$6n^2 + 3n + 10$ est dominée asymptotiquement par n^2".

<u>Exercice 1</u> :
Écrire un algorithme permettant de trouver le plus grand entier présent dans un tableau. Faire "tourner à la main" l'algorithme en utilisant le tableau t = [3,5,1,8,4,2]. Déterminer ensuite la complexité de l'algorithme.

Max ← t[0]	**affectation : 1**
n ← longueur de t	**affectation : 1**
pour i variant de 1 à n-1	**itérations : n – 1**
si Max <= t[i] alors Max ← t[i]	**test et affectation : au plus 2**
fin pour si	
fin	

Fonctionnement à la main avec t = [3,5,1,8,4,2] :

Max ← 3

n ← 6

i = 1 et i <= 6 on entre dans la boucle

3 < t[1] (c'est-à-dire 5) alors Max ← 5 et on passe à l'itération suivante

Max ← 5

n ← 6

i = 2 et i <= 6 on entre dans la boucle

5 > t[2] (c'est-à-dire 1) on passe à l'itération suivante

Max ← 5

n ← 6

i = 3 et i <= 6 on entre dans la boucle

5 < t[3] (c'est-à-dire 8) alors Max ← 8 et on passe à l'itération suivante

Max ← 8

n ← 6

i = 4 et i <= 6 on entre dans la boucle

8 > t[4] (c'est-à-dire 4) on passe à l'itération suivante

Max ← 8

n ← 6

i = 5 et i <= 6 on entre dans la boucle

8 > t[5] (c'est-à-dire 2) alors Max ← 8

Fin

On a une complexité en temps de T(n) = 2(n – 1) + 2 = 2n.
Soit une complexité en O(n).

<u>Exercice 2</u> :

Écrire un algorithme permettant de calculer la moyenne de tous les entiers présents dans un tableau. Faire "tourner à la main" l'algorithme en utilisant le tableau t = [3,5,1,8,4,2]. Déterminer ensuite la complexité de l'algorithme.

somme ← 0	**affectation : 1**
n ← longueur de t	**affectation : 1**
pour i allant de 0 à n – 1:	**itérations : n**
somme ← somme + liste[i]	**calcul + affectation : 2**
moyenne ← somme / n	**calcul + affectation : 2**

Fonctionnement à la main avec t = [3,5,1,8,4,2] :

somme ← 0

n ← 6

i = 0

somme ← somme + t[0] (c'est-à-dire 3)

somme = 3

n ← 6

i = 1

somme ← somme + t[1] (c'est-à-dire 5)

somme = 8

n ← 6

i = 2
  somme ← somme + t[0] (c'est-à-dire 1)

somme ← 9
n ← 6
i = 3
  somme ← somme + t[0] (c'est-à-dire 8)

somme ← 17
n ← 6
i = 4
  somme ← somme + t[0] (c'est-à-dire 4)

somme ← 21
n ← 6
i = 5
  somme ← somme + t[0] (c'est-à-dire 2)
fin pour
moyenne ← somme (c'est-à-dire 23) / 6

On a une complexité en temps de T(n) = 2n + 4.
Soit une complexité en O(n).

II) COMPLEXITE D'UN ALGORITHME :

Le calcul de la **complexité** d'un algorithme permet de mesurer sa **performance**. Il existe deux types de complexité :

- **complexité spatiale** : permet de quantifier l'utilisation de la **mémoire,**
- **complexité temporelle** : permet de quantifier la **vitesse** d'exécution.

Dans ce qui suit nous ne parlerons que de complexité temporelle.

1°) Complexité temporelle :

L'objectif d'un calcul de **complexité algorithmique temporelle** est de pouvoir comparer l'**efficacité** d'algorithmes résolvant le même problème. Dans une situation donnée, cela permet donc d'établir lequel des algorithmes disponibles est le plus **optimal**.
Pour des données volumineuses la différence entre les durées d'exécution de deux algorithmes ayant la même finalité peut être de l'ordre de plusieurs jours !
Réaliser un calcul de complexité en temps revient à compter le nombre d'**opérations élémentaires** (affectation, calcul arithmétique ou logique, comparaison…) effectuées par l'algorithme.
Puisqu'il s'agit seulement de comparer des algorithmes, les règles de ce calcul doivent être indépendantes :

- du langage de programmation utilisé ;
- du processeur de l'ordinateur sur lequel sera exécuté le code ;
- de l'éventuel compilateur employé.

Par soucis de simplicité, on fera l'hypothèse que toutes les opérations élémentaires sont à **égalité de coût**, soit 1 « unité » de temps.
*Exemple : a = b * 3 : 1 multiplication + 1 affectation = 2 « unités ».*

La **complexité** en temps d'un algorithme sera exprimée par une fonction, notée *T* (pour *Time*), qui dépend :

- de la taille des données passées en paramètres : plus ces données seront volumineuses, plus il faudra d'opérations élémentaires pour les traiter.
  On notera *n* le nombre de données à traiter.
- de la donnée en elle-même, de la façon dont sont réparties les différentes valeurs qui la constituent.
  par exemple, si on effectue une recherche séquentielle d'un élément dans une liste non triée, on parcourt un

par un les éléments jusqu'à trouver, ou pas, celui recherché. Ce parcours peut s'arrêter dès le début si le premier élément est « le bon ». Mais on peut également être amené à parcourir la liste en entier si l'élément cherché est en dernière position, ou même n'y figure pas.

Cette remarque nous conduit à préciser un peu notre définition de la complexité en temps. En toute rigueur, on peut en effet distinguer deux formes de complexité en temps :

- la complexité dans le **meilleur des cas** : c'est la situation la plus favorable,
  par exemple : recherche d'un élément situé à la première position d'une liste.
- la complexité dans le **pire des cas** : c'est la situation la plus défavorable,
  par exemple : recherche d'un élément dans une liste alors qu'il n'y figure pas.

On calculera le plus souvent la complexité dans le pire des cas, car elle est la plus pertinente. Il vaut mieux en effet toujours envisager le pire.

2°) Ordre de grandeur :

Pour comparer des algorithmes, il n'est pas nécessaire d'utiliser la fonction T, mais seulement l'ordre de grandeur asymptotique, noté O (« grand O »).

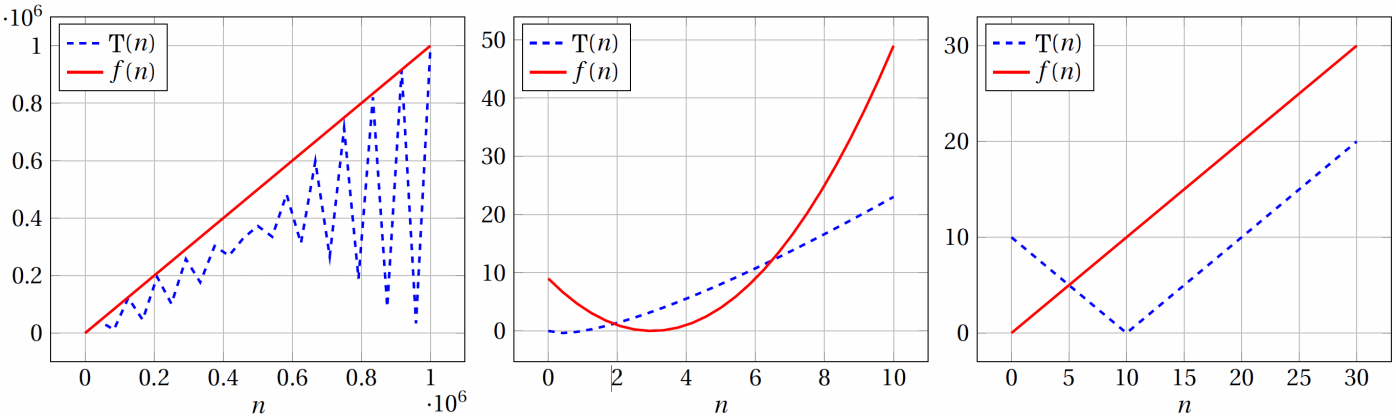
Une fonction $T(n)$ est en $O(f(n))$ (« *en grand O de f(n)* ») si : $\exists\, n_0 \in \mathbb{N}, \exists\, c \in \mathbb{R}, \forall\, n \geq n_0 : |T(n)| \leq c|f(n)|$.

Autrement dit :

$T(n)$ est en $O(f(n))$ s'il existe un seuil n_0 à partir duquel la fonction T est toujours dominée par la fonction f, à une constante multiplicative fixée c près.

Exemples :

- $T1(n)=7=O(1)$
- $T2(n)=12n+5=O(n)$
- $T3(n)=4n^2+2n+6=O(n^2)$
- $T4(n)=2+(n-1)\times 5=O(n)$

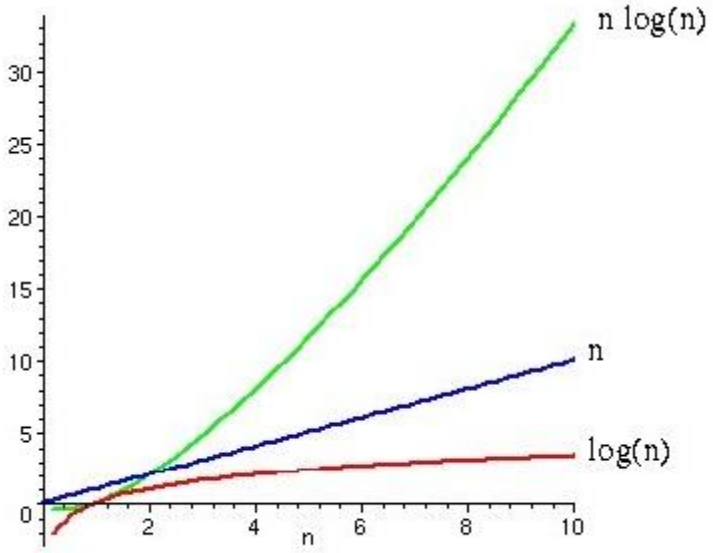


Classes de complexité :

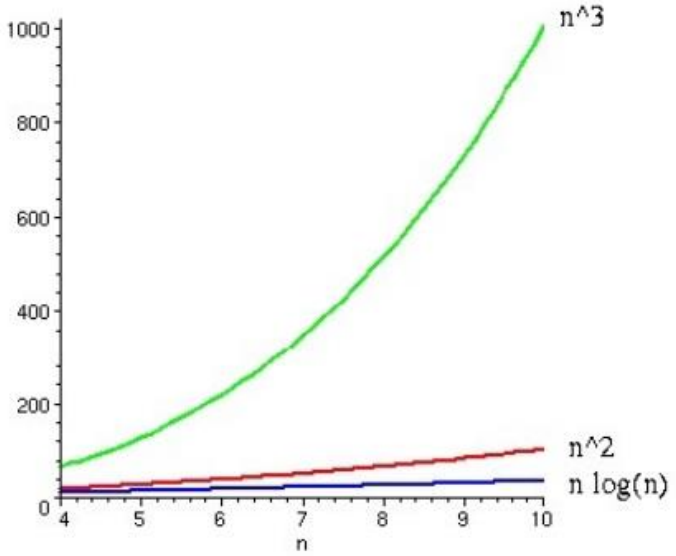
O	Type de complexité
$O(1)$	constante
$O(log(n))$	logarithmique
$O(n)$	linéaire
$O(n\times log(n))$	quasi-linéaire
$O(n^2)$	quadratique
$O(n^3)$	cubique
$O(2^n)$	exponentielle
$O(n!)$	factorielle

3°) Courbes évolutives associées :

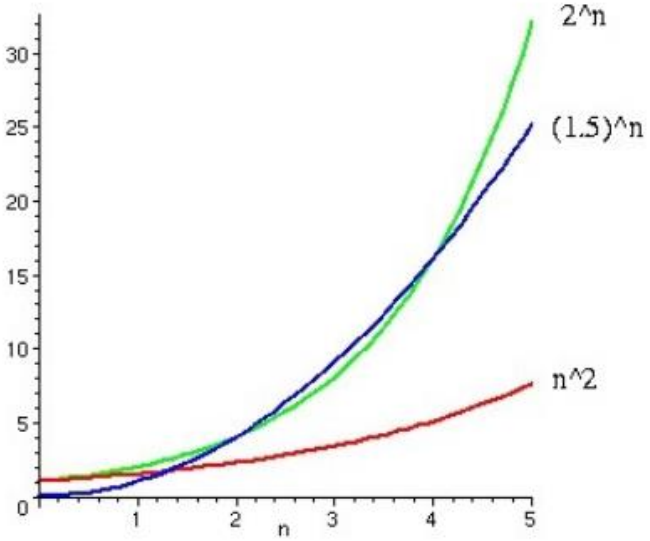
n, log(n), nlog(n)



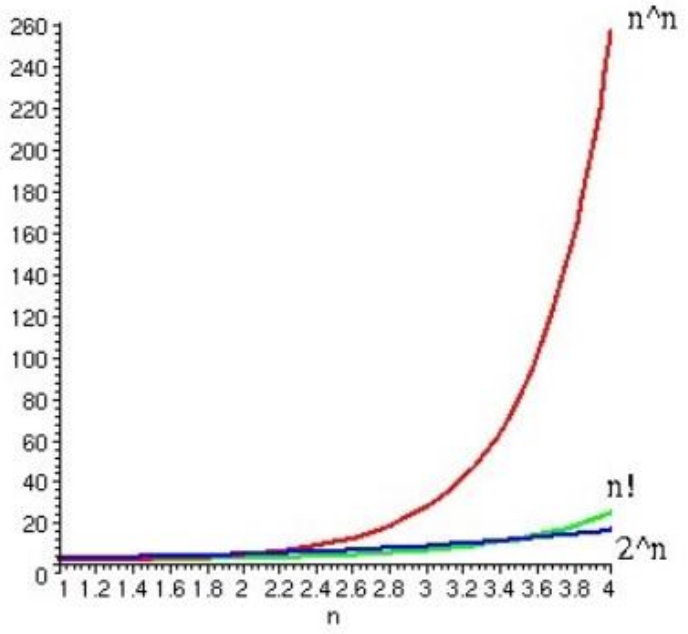
nlog(n), n^2, n^3



n^2, 1.5^n ,2^n



2n, n^n, n!



Exemple de calcul de complexité :

```
def factorielle(n):                    .

  fact = 1                             affectation : 1
  i = 2                                affectation : 1
                                       itérations : au plus n − 1
  while i <= n:                              comparaison : 1
    fact = fact * i                          multiplication + affectation : 2
    i = i + 1                                addition + affectation : 2
  return fact
```

On a donc une complexité en temps de T(n) = 1 + 1 + 5(n – 1) = 5n – 3.
Soit une complexité en : O(n).

4°) Exercices corrigés :

Exercice 1 :

```
def fonction_(n):
  for i in range(n):                   itérations : n
    print("Bonjour!")                        affichage : 1
```

On a donc une complexité en temps de T(n) = 1(n) = n (parcours de la boucle avec n répétitions).
Soit une complexité en O(n).

Exercice 2 :

```
n = 100                                affectation : 1
s = 0                                  affectation : 1
for i in range(n):                     itérations : n
  a = n                                      affectation : 1
  while a > 1:                               itérations : 8 pour arriver à a < 1 et comparaison : 1
    a = a/2                                  affectation + calcul : 2
  s += 1                                     addition + affectation : 2
```

On a une complexité en temps de T(n) = 1 + 1 + ((8 + 1 + 2 + 2)(n)) = 13n + 2.
Soit une complexité en O(n).

5°) Exercices :

Calculer la complexité des fonctions ci-dessous :

Exercice 1 :

```
def conversion(n):
    h = n // 3600                    division euclidienne + affectation : 2
    m = (n - 3600*h) // 60           soustraction + multiplication + division euclidienne+ affectation : 4
    s = n % 60                       modulo + affectation : 2
    return h,m,s
```

On a une complexité en temps de T(n) = 8.
Soit une complexité en O(1).

Exercice 2 :

```
def puissanceMoinsUn(n):
  if n%2==0:                         calcul modulo + test : 2
    res = 1                          affectation : 1
  else:
    res = -1
  return res
```

On a une complexité en temps de T(n) = 3.
Soit une complexité en O(1).

Exercice 3 :

```
def sommeEntiers(n):
    somme = 0                        affectation : 1
    for i in range(n+1):             itérations : n+1
        somme += i                           affectation + calcul : 2
    return somme
```

On a une complexité en temps de T(n) = 2(n + 1) + 1 = 2n + 3.
Soit une complexité en O(n).

Exercice 4 :

```
def factorielle(n):
  fact = 1                           affectation : 1
  i = 2                              affectation : 1
  while i <= n:                      itérations : n – 1 + comparaison : 1
    fact = fact * i                          calcul + affectation : 2
    i = i + 1                                calcul + affectation : 2
  return fact
```

On a une complexité en temps de T(n) = 2 + 5(n – 1) = 5n – 3.
Soit une complexité en O(n).

Exercice 5 :

```
def triSelection(l):
    for i in range(len(l)-1):        itérations : n – 1 avec n = len(l)
        indMini=i                            affectation : 1
        for j in range(i+1,len(l)):          itérations : en moyenne n/2
            if l[j]<l[indMini]:                      comparaison : 1
                indMini=j                            affectation : 1
        l[i],l[indMini]=l[indMini],l[i]       permutations : 2
```

On a une complexité en temps de T(n) = (n-1)(2(n/2) + 3) = (n – 1)(n + 3) = n^2 + 2n – 3.
Soit une complexité en O(n^2).

III) CONCLUSION :

Déduire des exemples de cette activité les règles suivantes :

Dans le cas d'un algorithme simple avec une boucle unique, la complexité est en O(n).
Dans le cas d'un algorithme simple avec deux boucles imbriquées, la complexité est en O(n^2).